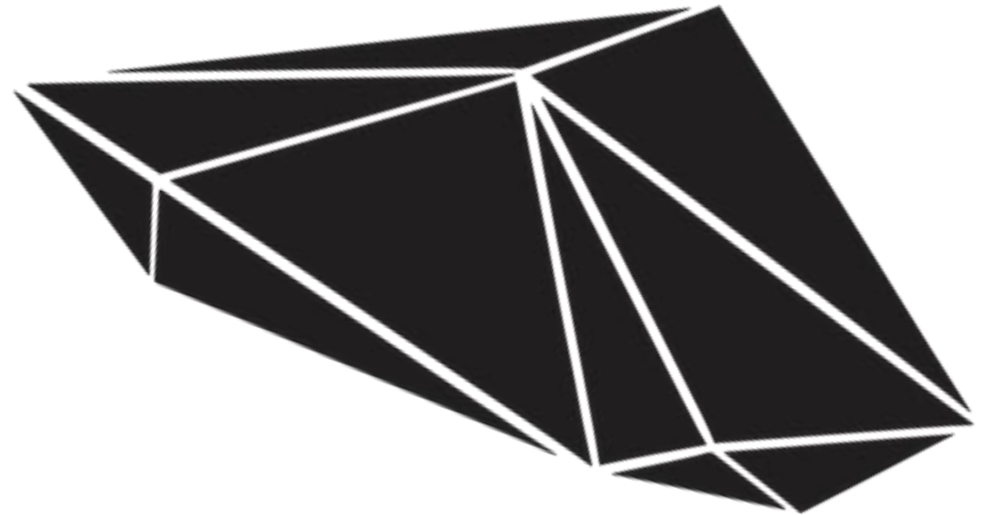

CP431 Group 9

Parallel Data Mining





Introduction

- Set out to mine frequent item-sets and compute association rules using the Apriori data mining algorithm and using parallel programming
- Program can be used for datasets of different sizes and has been tested using sets of thousands of instances.
- During our research we discovered the SON (Savasere, Omiecinski, and Navathe) algorithm. This itemset mining algorithm can use several different algorithms as its base (such as PCY or multihash). We decided that the best approach would be to implement this algorithm, using Apriori as the basic itemset finder.



Why Parallelize Association Rule Mining?

- Frequent itemset mining (e.g. market basket analysis) becomes slow with large datasets
- Apriori needs multiple passes over data, which becomes a bottleneck
- File I/O and support counting are expensive operations
- Goal: **speed it up with parallelism using MPI**
 - expensive IO operations to be evenly distributed among many processors

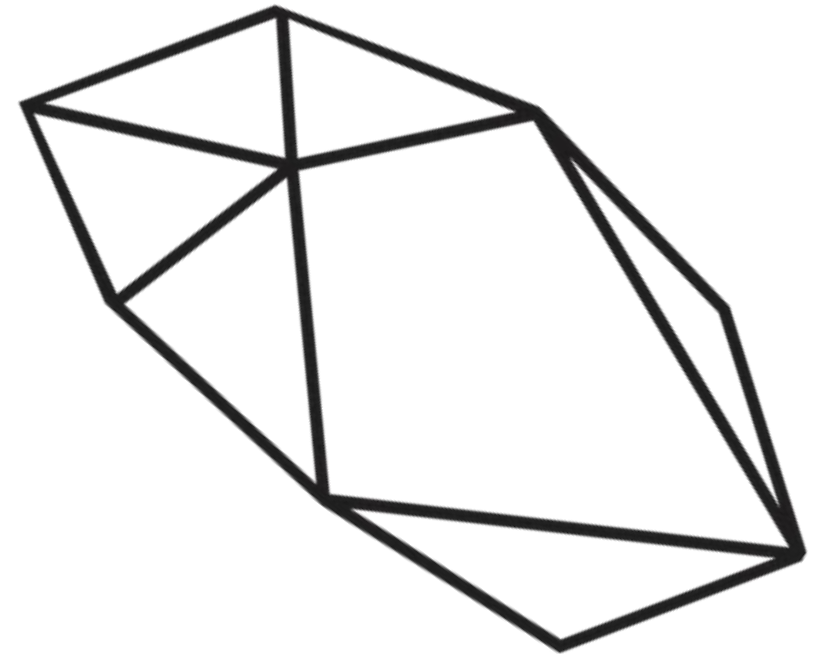


Our Approach

- Used SON algorithm (parallel Apriori framework)
- Split the data across p processes using MPI
- Each process finds frequent items locally, merges the results, and generates candidate sets and rules
- We focused on efficiency and scalability

System Design & Tools

- Language used: C
- Parallelization: MPI (tested on UofT Teach cluster)
- Structures used:
 - Dynamic hash tables for itemsets
 - Triangular matrix for pairs (saves memory).
- Tools created:
 - map_items.c: text to numeric preprocessing
 - map_lookup.c: numeric rules to item names



Dataset Handling & Preprocessing

- ds1-ds3 - Numeric transactions
- Ds4 - Real item names (mapped)
- Ds5 - Large set (1M + transactions)
- Text Input preprocessed to numeric form (`map_items.c`)
- Human-readable rules generated with `map_lookup.c`

Example:

`{39, 17}` $\longrightarrow$ `{20}` $\longrightarrow$ `{jams-spreads, milk-cream}` $\longrightarrow$ `{sauces-gravy-pkle}`



Implementation Steps (SON + Apriori)

1. Input: File split among process
2. Frequent Items: Local support counted
3. Pairs: Triangular matrix updated + pruned
4. Larger Itemset: size k-itemsets generated and filtered
5. Rule Generation: Master creates rules meeting confidence

Results on ds5 (1M+ transactions)

- Support = 50%
- Confidence = 75%

Top Rule:

- {36, 39} → {38}
 - Confidence: 95.19%
 - Support: 2.38%

Data-set	# of Transactions	Par. Time (30 cores)
ds1	88,161	126.240
ds5	1,057,944	12.264

Example output (Grocery Store)

Top 10 association rules:

Rule : {citrus, bread} => {fruit} (Confidence: 90.00%, Support: 1.37%)

Rule : {citrus, margarine} => {fruit} (Confidence: 100.00%, Support: 0.75%)

Rule : {citrus, tropical-fruit} => {fruit} (Confidence: 100.00%, Support: 1.99%)

Rule : {fruit, curd} => {whole-milk} (Confidence: 81.33%, Support: 1.32%)

Rule : {citrus, yogurt} => {fruit} (Confidence: 100.00%, Support: 2.17%)

Rule : {citrus, coffee} => {fruit} (Confidence: 100.00%, Support: 0.62%)

Rule : {citrus, whole-milk} => {fruit} (Confidence: 100.00%, Support: 3.05%)

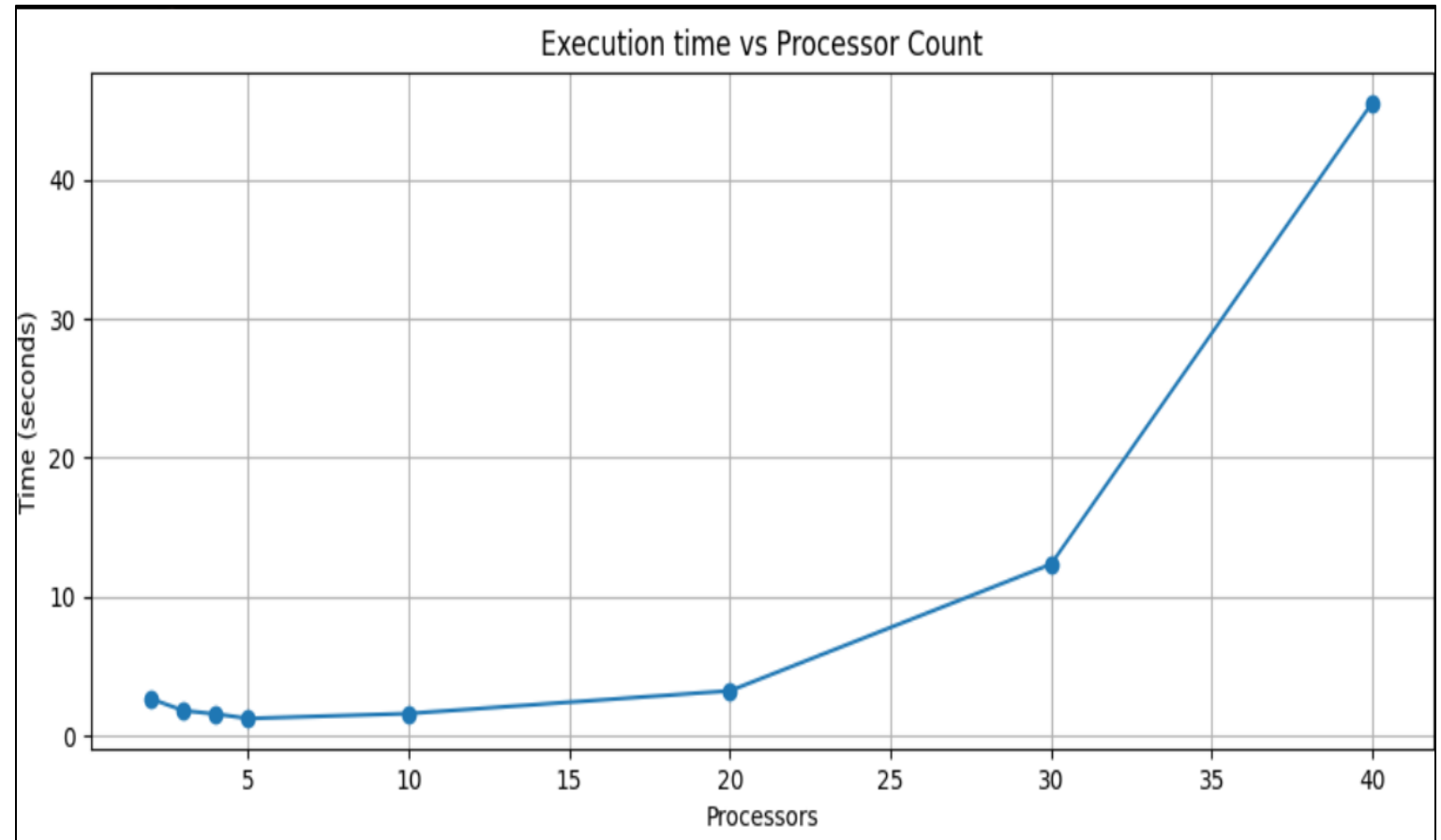
Rule : {fruit, tropical-fruit} => {whole-milk} (Confidence: 77.55%, Support: 1.55%)

Rule : {cream, eggs} => {whole-milk} (Confidence: 83.42%, Support: 1.12%)

Rule {citrus, cream} => {fruit} (Confidence: 90.24%, Support: 1.50%)

Performance Analysis

- Most of execution time spent on large itemset mining
- Apriori becomes computationally expensive as k increases
- Optimal performance at 5 processors
- Using a triangular matrix reduces storage
- Dynamic hash table improves lookup efficiency





Challenges and Improvements

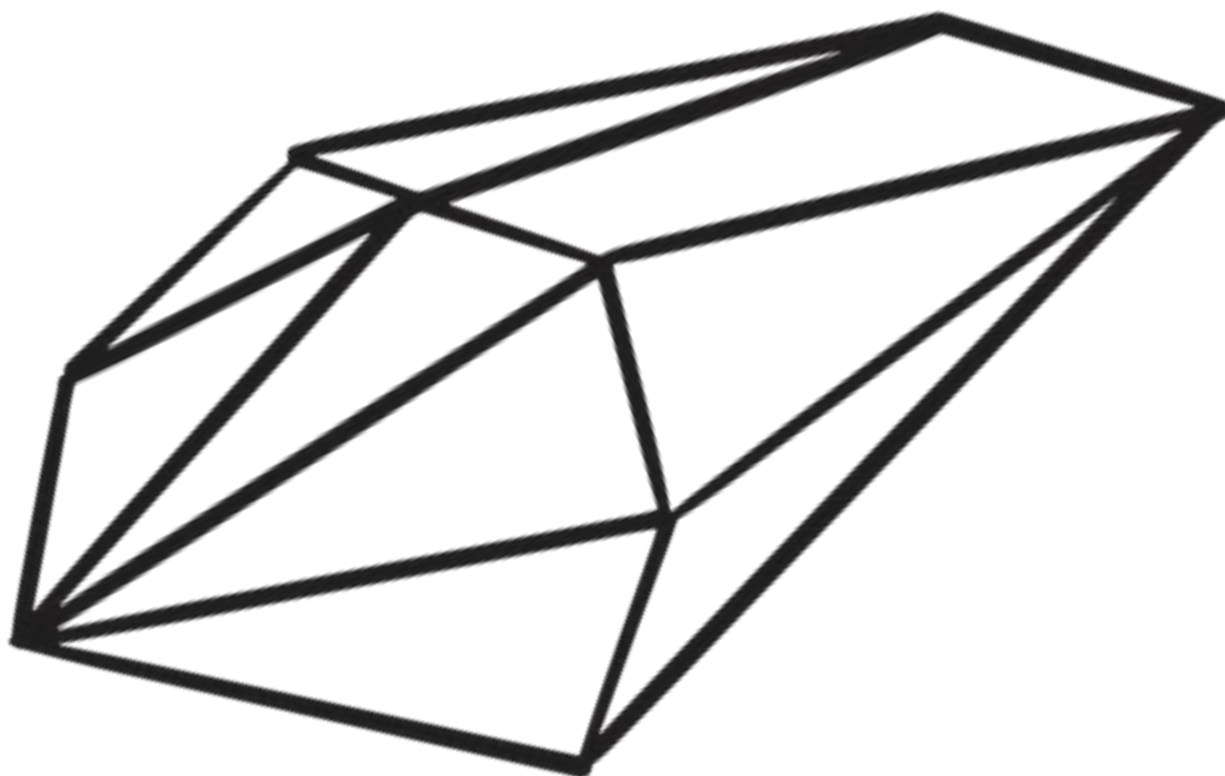
- Initial bugs in triangular matrix indexing
- Limited by dataset size vs available memory
- MPI comm overhead limits scaling past 32 cores

Future ideas:

- Compare with FP-Growth
- Try pure shared-memory with OpenMP



Thank you!



Introduction

We set out to **mine frequent item-sets and compute association rules** using the Apriori data mining algorithm from a parallel approach. Our program can be used for datasets of different sizes and has been tested using sets of thousands of instances. **K+1 passes** (k is the size of the largest frequent itemset present) must be made over the dataset before rule generation can take place in Apriori. Taking advantage of data parallelism and implementing Apriori on a high-performance compute cluster allows for operations to be evenly distributed among many processors, leading to **massive improvements in efficiency**.

System Design

We used **C with MPI** for this project, and **UofT Teach** as a high-performance cluster for a parallel computing environment. We used **hash tables** for the frequently occurring terms, and a **triangular matrix** for counting item pairs.

Datasets Used:

- **ds1–ds3**: Numeric item-based datasets with varied transaction sizes
- **ds4**: Real-world item names requiring preprocessing
- **ds5**: Stress test dataset with over 1,000,000 transactions

Dataset Handling:

- Input: transactions stored in text files, one per line
- Supported text-based and numeric datasets
- Real-world datasets ds4.txt required preprocessing to create ds4_numeric.txt and map.txt
- Example:

Before Mapping	After Mapping (using map_lookup.c)
{39, 17} -> {20}	{jams-spreads, milk-cream} -> {sauces-gravy-pkle}

Implementation

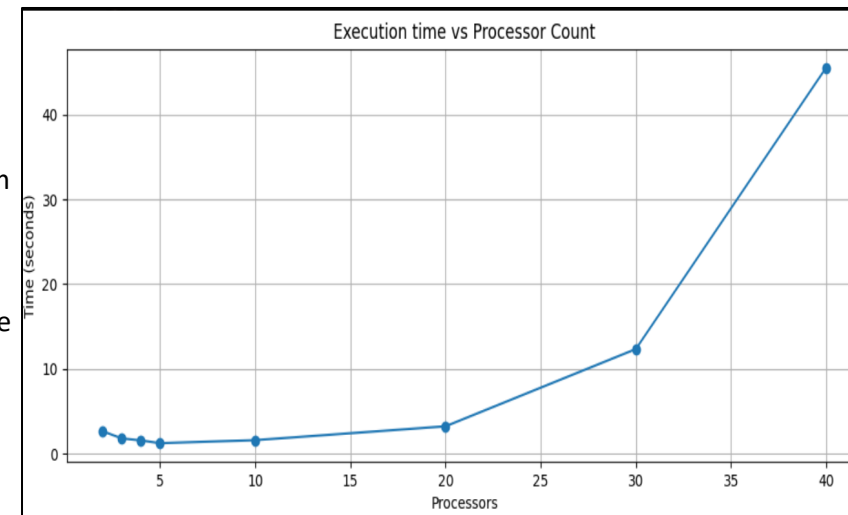
1. The **input file** - Converted to a numeric format and split up based on file size, and a **hashtable with frequent terms** is created.
2. The **SON algorithm** is implemented, which parallelizes Apriori. The master process determines the chunks and distributes them using **MPI_Bcast()**. Each process counts and filters item occurrences locally. The total transaction count is then sent to the master process.
3. **Pair Mining** – the processes scan their chunk again, updating the item pair counts in the triangular matrix, which is then pruned.
4. Itemset/ Rule Generation – frequent pairs are used to generate larger itemsets then pruned. The master process gathers frequent itemsets from all processes and generates association rules based on **confidence**.

Example Results

We tested our MPI-based Apriori implementation on multiple datasets to evaluate performance and rule generation.
ds5 (1M+ transactions, 30 cores, support = 5%, confidence = 75%)
736 rules generated
Top rule: {36, 39} → {38} (95.19% confidence, 2.38% support)
Execution Time: 12.264 seconds

Data-set	# of Transactions	Time (30 cores)
ds1	88,161	126.240
ds5	1,057,944	12.264

Time Analysis



- Most of execution time was spent on large itemset mining
- Became more computationally expensive as k increased
- Optimal performance at 5 processors, believed to be because the program did not scale well into high number of processors due to how local support is handled
- Processors finding more frequent itemsets larger than 2 and spent more time generating

Conclusion

- MPI-based Apriori significantly reduces execution time, achieving near-linear speedup on large datasets.
- Data partitioning and parallel pruning allow for efficient frequent itemset discovery.
- Parallelism improves performance, though diminishing returns appear when dataset size does not scale with processor count
- Future Work: Explore FP-Growth for even better efficiency in large-scale itemset mining.